

# **Antioquia Biodiversa**

## **Propuesta de Ajustes Técnicos Versión 2.**

Gobernación de Antioquia - Secretaría de Ambiente

Junio 2026

**Versión:** v2.0 - Actualizada con respuestas del equipo de TI

## Contexto y alcance del documento

Este documento recoge los ajustes técnicos acordados para el proyecto Antioquia Biodiversa a partir de la revisión de la propuesta original y las respuestas recibidas del equipo de TI de la Gobernación de Antioquia. El stack base (Node.js 22 LTS, MongoDB Atlas, arquitectura modular con Express) se mantiene sin cambios. Los cinco ajustes descritos a continuación corresponden a mejoras en seguridad, rendimiento, observabilidad y continuidad operativa.

Esta versión incorpora dos precisiones importantes frente al documento anterior:

- **SAST en dos capas:** El análisis estático de seguridad (SAST) se implementa en dos capas: **ESLint-security (filtro rápido en el commit)**, **Semgrep Community con reglas OWASP (análisis profundo)**.
- **CI/CD on-premises:** El pipeline de Integración Continua (CI) se configurará usando las plantillas ya generadas por el equipo de TI. El Despliegue Continuo (CD) depende del esquema de gestión de servidores on-premises que maneja la Gobernación.

## Resumen de cambios

La siguiente tabla relaciona cada ajuste con los requisitos no funcionales (RNF) del levantamiento de requerimientos que impacta:

| Ajuste                                               | RNF impactado | Mejora concreta                                                     |
|------------------------------------------------------|---------------|---------------------------------------------------------------------|
| 1. Microsoft Entra ID                                | RNF05, RNF08  | Elimina contraseñas en la BD. MFA automático. Revocación inmediata. |
| 2. Caché con Redis 7                                 | RNF02, RNF06  | Catálogo en <5 ms. MongoDB libre para escrituras bajo carga alta.   |
| 3. Monitoreo (Winston + Loki + Prometheus + Grafana) | RNF06, RNF07  | SLA 99% medible con evidencia. Alertas proactivas antes de fallas.  |
| 4. SAST (ESLint-security + Semgrep)                  | RNF05         | OWASP Top 10 verificado en 2 capas en cada push.                    |
| 5. Política de backup (MongoDB Atlas Snapshots)      | RNF06, RNF07  | RPO ≤ 24 h. RTO < 4 h. Protección ante pérdida de datos.            |

## 1. Autenticación con Microsoft Entra ID

### 1.1 Situación actual

El panel de administración utiliza express-session con bcrypt. Las contraseñas de los curadores se almacenan en la base de datos de la aplicación (MongoDB Atlas), lo que crea dos riesgos: (a) si la base de datos es comprometida, un atacante obtiene las credenciales de acceso al panel, y (b) la gestión del ciclo de vida de los usuarios (altas, bajas, cambios de contraseña) recae sobre el contratista durante toda la garantía.

## 1.2 Propuesta

Integrar el panel de administración con Microsoft Entra ID (Azure Active Directory institucional de la Gobernación). Los curadores iniciarán sesión con su cuenta corporativa @antioquia.gov.co, la misma que usan para el correo y Microsoft Teams. La aplicación nunca almacena ni verifica contraseñas: solo valida el token que emite Microsoft.

## 1.3 Justificación

- **Infraestructura existente:** La Gobernación ya tiene Entra ID con todos sus funcionarios registrados. No se monta un sistema de identidad paralelo.
- **MFA sin desarrollo extra:** El Multifactor Authentication (MFA) vía Microsoft Authenticator, ya activo para los funcionarios, se hereda sin costo adicional de desarrollo.
- **Revocación instantánea:** Cuando un curador sale de la institución, el área de TI desactiva su cuenta en Entra ID y pierde el acceso al panel de forma inmediata y automática.
- **Superficie de ataque reducida:** La aplicación nunca toca una contraseña. Si la base de datos es comprometida, no hay credenciales de acceso al panel que extraer.

## 1.4 Especificación técnica

| Componente                      | Detalle                                                         |
|---------------------------------|-----------------------------------------------------------------|
| Protocolo                       | OAuth 2.0 + OpenID Connect (OIDC)                               |
| Flujo de autorización           | Authorization Code Flow con PKCE                                |
| Librería backend (Node.js)      | @azure/msal-node + passport-azure-ad                            |
| Librería frontend (panel admin) | @azure/msal-browser                                             |
| Sesión                          | JWT emitido por Entra ID — sin contraseñas en la aplicación     |
| MFA                             | Heredado del tenant institucional (Microsoft Authenticator)     |
| Roles de aplicación             | Curador.Biodiversidad · Curador.GuardaCuentas · Admin.Contenido |
| Definición de roles             | App Registration en el tenant de la Gobernación                 |

## 1.5 Prerrequisitos: acción requerida por la Gobernación

Para iniciar el desarrollo de este ajuste, el área de TI de la Gobernación debe proveer lo siguiente:

- Client ID de la aplicación registrada en el tenant de Entra ID.
- Tenant ID del directorio institucional.
- Configuración de los redirect\_uri para cada ambiente: QA (<http://localhost:3000/auth/callback>) y Producción (<https://biodiversa.antioquia.gov.co/auth/callback>).
- Definición de los roles en el App Registration: Curador.Biodiversidad, Curador.GuardaCuentas, Admin.Contenido.

## 2. Caché con Redis 7

### 2.1 Situación actual

Cada solicitud al catálogo de especies genera una consulta directa a MongoDB Atlas. En condiciones normales, el tiempo de respuesta es de 60–120 ms. En eventos de alto tráfico (escaneo masivo de QR), múltiples peticiones simultáneas saturan la base de datos sin beneficio real, dado que los datos del catálogo cambian únicamente cuando el administrador realiza una importación masiva.

### 2.2 Propuesta

Implementar Redis 7 como capa de caché entre Express y MongoDB, usando la librería ioredis. La primera consulta a cada endpoint llena el caché; las solicitudes subsiguientes se resuelven desde memoria RAM en menos de 5 ms. Redis se ejecuta en el mismo servidor Ubuntu en modo standalone, sin costo de infraestructura adicional.

### 2.3 Estrategia de TTL e invalidación

| Recurso / Endpoint                   | TTL        | Evento de invalidación                        |
|--------------------------------------|------------|-----------------------------------------------|
| /api/especies (catálogo completo)    | 30 minutos | El admin ejecuta importación masiva           |
| /api/especies/:id (ficha individual) | 60 minutos | Se actualiza esa especie en la BD             |
| /api/subregiones y /api/grupos       | 24 horas   | Dato estático — invalidación manual           |
| Galería JPL / Guarda Cuencas         | 10 minutos | Se publica el JSON mensual desde el panel     |
| Especie del Mes                      | 1 hora     | Se actualiza desde el panel de administración |

### 2.4 Especificación técnica

| Componente         | Detalle                                                     |
|--------------------|-------------------------------------------------------------|
| Software           | Redis 7.x (paquete oficial de Ubuntu)                       |
| Librería Node.js   | iredis ^5                                                   |
| Modo de despliegue | Standalone en el servidor de aplicación (sin cluster)       |
| Memoria máxima     | 256 MB con política de evicción allkeys-lru                 |
| Persistencia       | RDB snapshot cada 15 min (tolerancia a reinicios)           |
| Monitoreo          | Métricas de hit/miss expuestas a Prometheus vía prom-client |

## 3. Monitoreo y observabilidad

### 3.1 Situación actual

Los únicos registros de actividad disponibles son los logs de PM2 en archivos de texto plano. No existen métricas de rendimiento, dashboards operativos ni alertas automatizadas. El equipo se entera de los fallos cuando un usuario lo reporta.

### 3.2 Propuesta — stack de observabilidad

Se implementa un stack operativo liviano que corre en el servidor de aplicación, compuesto por cuatro componentes:

| Componente               | Función                                                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| Winston                  | Logs JSON estructurados con nivel, módulo, timestamp y traceId único por petición.                  |
| Loki                     | Almacenamiento y consulta de logs. Permite buscar por traceId, nivel o módulo desde Grafana.        |
| Prometheus + prom-client | Métricas HTTP en tiempo real: latencia p50/p95/p99, tasa de errores, throughput, hit-rate de Redis. |
| Grafana                  | Dashboard operativo con visualización de logs y métricas. Alertas configurables.                    |
| GET /api/health          | Endpoint que verifica MongoDB y Redis. Usado por Nginx como upstream health check.                  |

### 3.3 Alertas configuradas

| Condición                                | Umbral                    | Canal de notificación    |
|------------------------------------------|---------------------------|--------------------------|
| Latencia p95 > 3 s en cualquier endpoint | >3 000 ms sostenido 5 min | Correo + Microsoft Teams |
| Tasa de errores HTTP 5xx                 | >5 % en ventana de 5 min  | Correo + Microsoft Teams |
| Espacio en disco del servidor            | <20 % libre               | Correo                   |
| Proceso PM2 de la aplicación caído       | Inmediato                 | Correo + Microsoft Teams |
| Redis no disponible                      | Inmediato                 | Correo                   |

### 3.4 Justificación — SLA 99 %

El RNF06 establece un SLA de disponibilidad del 99 % mensual (máximo 7.2 horas de inactividad al mes). Sin métricas, ese compromiso es una promesa que no se puede demostrar ni auditar. Con Grafana, el porcentaje de disponibilidad queda registrado como evidencia verificable en cualquier acta de entrega o revisión contractual.

## 4. Análisis estático de seguridad (SAST) en Azure DevOps

### 4.1 Situación actual

El RNF05 establece cumplimiento con el OWASP Top 10, pero no existe ningún mecanismo automatizado que lo verifique. La detección de vulnerabilidades depende exclusivamente de la revisión manual durante el pull request, que no es sistemática ni reproducible.

### 4.2 Propuesta — dos herramientas complementarias

Se integran ESLint-security y Semgrep Community en el pipeline de Azure DevOps. Las dos herramientas se ejecutan automáticamente en cada push y el pipeline falla si se detecta una vulnerabilidad de severidad Alta o Crítica.

### 4.3 Herramientas — dos herramientas complementarias

Cada herramienta cumple un rol distinto. No se solapan: se complementan.

| Herramienta       | Rol en el pipeline                                              | Alcance                                                                                                                      | Costo                  |
|-------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------|
| ESLint-security   | Filtro rápido — corre en segundos, falla el build de inmediato. | Patrones peligrosos específicos de Node.js/Express: eval() con input de usuario, path traversal, prototype pollution, ReDoS. | Gratuito. Plugin npm.  |
| Semgrep Community | Análisis OWASP profundo — corre después de ESLint.              | Reglas p/nodejs y p/owasp-top-ten. Cubre las 10 categorías OWASP para Node.js/JavaScript.                                    | Gratuito. Binario CLI. |

### 4.4 Flujo en el pipeline de CI

El pipeline de Azure DevOps ejecuta los siguientes pasos en orden:

- **Paso 1:** Checkout del código fuente desde Azure Repos.
- **Paso 2:** npm install - instalación de dependencias.
- **Paso 3:** npm audit — análisis de dependencias de terceros. Falla el pipeline si detecta vulnerabilidades conocidas de severidad Alta o Crítica (CVEs).
- **Paso 4:** npm run lint - ESLint con eslint-plugin-security. Falla el pipeline si detecta una regla de severidad error.
- **Paso 5:** npm test - ejecución de pruebas unitarias.
- **Paso 6:** Semgrep scan - análisis con reglas p/nodejs y p/owasp-top-ten. Falla el pipeline si detecta vulnerabilidad de severidad Alta o Crítica.
- **Paso 7:** (Solo en rama develop/main) Build y empaquetado para despliegue.

## 5. Política de backup

### 5.1 Situación actual

No existe una política formal de backup. Ante una falla de hardware, corrupción de datos o eliminación accidental por parte de un curador, la información no tiene mecanismo de recuperación. Esto afecta tanto el catálogo de especies (resultado de investigación científica) como las fotografías y registros de los programas JPL y Guarda Cuencas (activos comunitarios).

### 5.2 Propuesta

Activar los snapshots automáticos diarios incluidos en los tiers de pago de MongoDB Atlas. La configuración no requiere software adicional: Atlas ya tiene la funcionalidad integrada y la activación es un cambio en el panel de administración del cluster.

### 5.3 Parámetros de la política

| Parámetro                      | Valor                           | Descripción                                                                               |
|--------------------------------|---------------------------------|-------------------------------------------------------------------------------------------|
| Frecuencia de snapshot         | Diaria (00:00 UTC-5)            | Una copia completa de ambas bases de datos cada 24 horas.                                 |
| Retención                      | 30 días                         | Se mantienen los últimos 30 snapshots. El más antiguo se elimina automáticamente.         |
| RPO (Recovery Point Objective) | ≤ 24 horas                      | Pérdida máxima de datos ante una falla: el trabajo de un día.                             |
| RTO (Recovery Time Objective)  | < 4 horas                       | Tiempo máximo desde la falla hasta la restauración completa de la app.                    |
| Alcance                        | BD biodiversidad + BD comunidad | Ambos clusters de MongoDB Atlas.                                                          |
| Verificación                   | Mensual                         | El contratista realiza una restauración de prueba en entorno QA y documenta el resultado. |

### 5.4 Justificación

- **Valor del dato:** El catálogo de 150 especies y sus fichas bilingües representan horas de investigación y validación científica que no son reproducibles fácilmente.
- **Activos comunitarios:** Las fotografías de Jóvenes pa' Lante y Guarda Cuencas son activos comunitarios con alto valor simbólico para los participantes del programa.
- **Costo marginal:** Atlas ya incluye la funcionalidad. El costo adicional es marginal y el riesgo que cubre es significativo.
- **Protección contractual:** Define claramente la responsabilidad ante pérdida de datos durante la garantía.

## Prerrequisitos y plan de implementación

## Prerrequisitos por parte de la Gobernación

| Prerrequisito                                                                                                            | Para qué ajuste     | Responsable    | Estado    |
|--------------------------------------------------------------------------------------------------------------------------|---------------------|----------------|-----------|
| Crear proyecto en Azure DevOps y proveer acceso al repositorio                                                           | Todos               | TI Gobernación | Pendiente |
| Proveer Client ID, Tenant ID y configurar redirect_uri en Entra ID                                                       | Ajuste 1 - Entra ID | TI Gobernación | Pendiente |
| Definir roles en App Registration (Curador.Biodiversidad, etc.)                                                          | Ajuste 1 - Entra ID | TI Gobernación | Pendiente |
| Confirmar VM/servidor on-premises y credenciales de despliegue                                                           | Ajustes 2, 3 y CD   | TI Gobernación | Pendiente |
| Confirmar y costear el tier de MongoDB Atlas M10+ para snapshots automáticos (costo operativo a cargo de la Gobernación) | Ajuste 5 - Backup   | TI Gobernación | Pendiente |

## Secuencia de implementación sugerida

Se estima la siguiente secuencia, considerando las dependencias entre ajustes y los tiempos de gestión de la Gobernación:

| Fase                        | Ajuste                                             | Duración estimada | Dependencia                          |
|-----------------------------|----------------------------------------------------|-------------------|--------------------------------------|
| Fase 0 — Infraestructura    | Azure DevOps: repositorio, tableros, plantillas CI | 1 semana          | Acceso a Azure DevOps                |
| Fase 1 — Calidad de código  | Ajuste 4: ESLint-security + Semgrep en pipeline CI | 2–3 días          | Acceso a Azure DevOps                |
| Fase 1 — Seguridad de datos | Ajuste 5: activar snapshots en MongoDB Atlas       | 1 día             | Tier Atlas M10+                      |
| Fase 2 — Rendimiento        | Ajuste 2: Redis 7 + ioredis en el backend          | 3–4 días          | VM/servidor definido                 |
| Fase 2 — Observabilidad     | Ajuste 3: Winston + Loki + Prometheus + Grafana    | 3–4 días          | VM/servidor definido                 |
| Fase 3 — Autenticación      | Ajuste 1: Microsoft Entra ID                       | 1–2 semanas       | Client ID + Tenant ID de Gobernación |

*Nota: Los ajustes 2 (Redis) y 3 (Monitoreo) pueden implementarse en paralelo una vez que el servidor on-premises esté disponible.*

## Stack tecnológico actualizado (v2)

| Capa                 | Tecnología                                    | Versión      | Estado                             |
|----------------------|-----------------------------------------------|--------------|------------------------------------|
| Frontend             | HTML5 + CSS3 + JavaScript Vanilla             | —            | Sin cambios                        |
| Backend              | Node.js + Express                             | 22 LTS / 4.x | Sin cambios                        |
| Base de datos        | MongoDB Atlas + Mongoose ODM                  | 7.x          | Sin cambios                        |
| Caché                | Redis + ioredis                               | 7.x / ^5     | Nuevo — Ajuste 2                   |
| Autenticación admin  | Microsoft Entra ID (OAuth 2.0 + OIDC)         | MSAL Node v3 | Reemplaza express-session + bcrypt |
| Logs                 | Winston + Loki                                | latest       | Nuevo — Ajuste 3                   |
| Métricas             | Prometheus + prom-client + Grafana            | latest       | Nuevo — Ajuste 3                   |
| SAST — capa 1        | ESLint-security (eslint-plugin-security)      | plugin npm   | Nuevo — Ajuste 4                   |
| SAST — capa 2        | Semgrep Community (p/nodejs, p/owasp-top-ten) | CLI gratuito | Nuevo — Ajuste 4                   |
| CI/CD                | Azure DevOps (plantillas TI Gobernación)      | —            | Nuevo — Fase 0                     |
| Servidor web         | Nginx + PM2                                   | latest       | Sin cambios                        |
| Imágenes             | sharp (WebP auto-conversión)                  | ^0.34        | Sin cambios                        |
| Uploads              | multer                                        | ^1.4         | Sin cambios                        |
| Control de versiones | Git + Azure Repos                             | —            | Sin cambios                        |

*Este documento está sujeto a revisión y aprobación por parte del equipo de TI y el líder funcional de la Gobernación de Antioquia antes de iniciar la implementación de cualquier ajuste.*